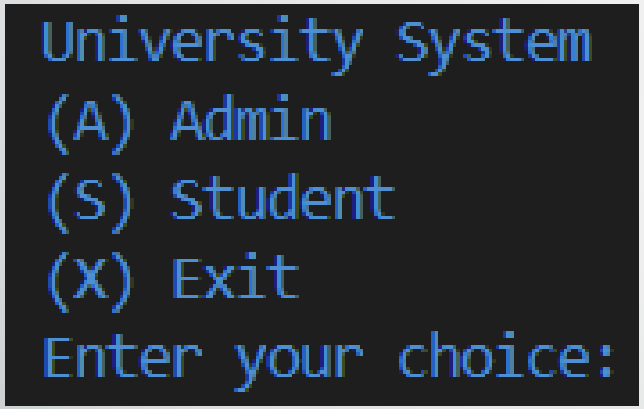
Enrollment System

Subject Coordinator: Dr. Yining Hu
Tutor: Dr. Iman Rahimi
Group: Cmp1 01 6

Student Name	ID
Ngoc Manh Nguyen	26312049
James An	11393372
Santhosh Ramachandran	26053850
Yanfei Tao	26295509

RESULT

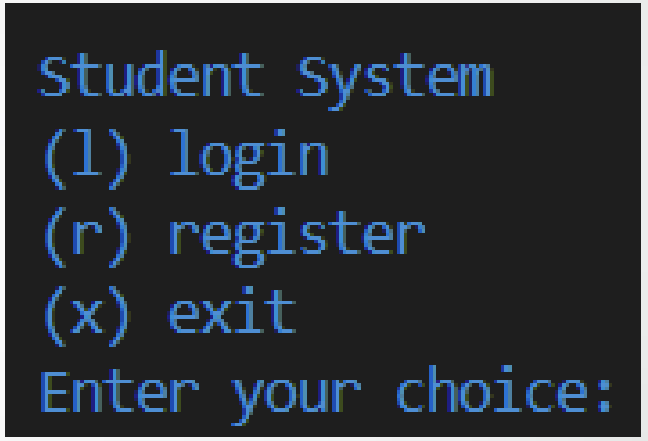


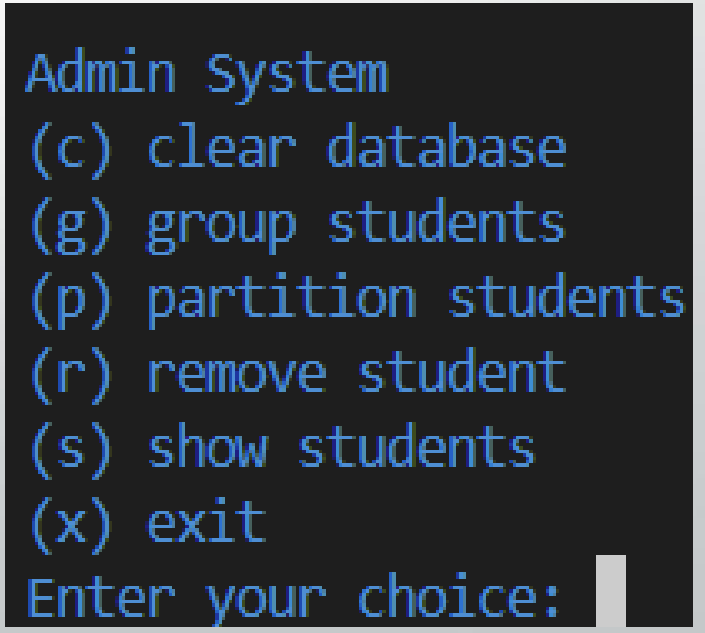
Starting menu

s

a

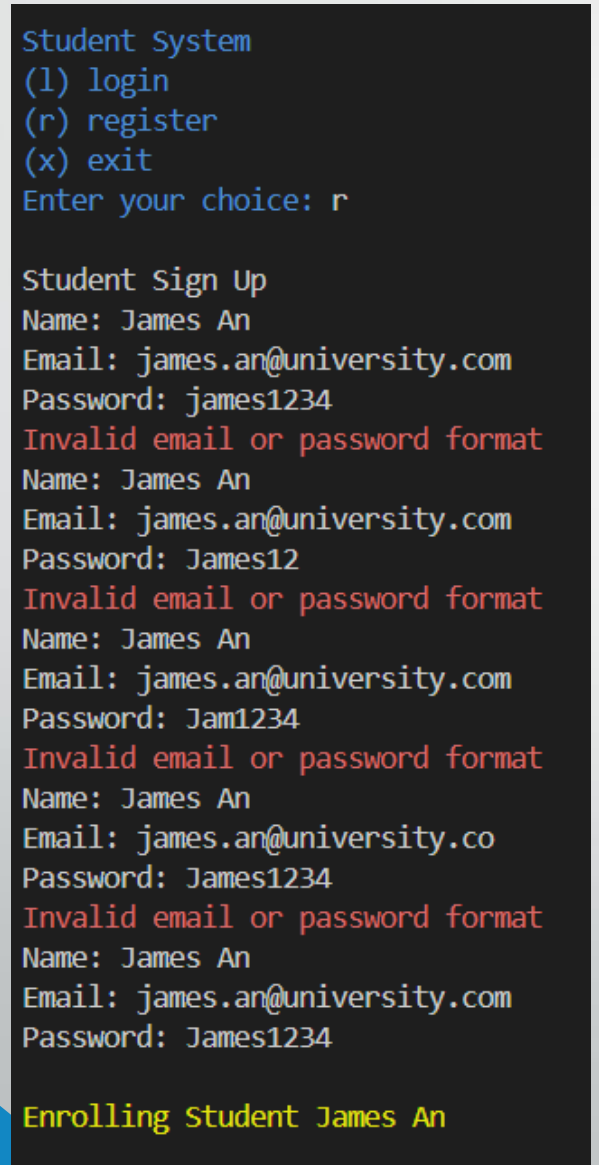
Student menu





Admin menu

RESULT – Student - Registration



Wrong password format (not start with an uppercase character)

Wrong password format (not enough 3 number)

Wrong password format (not enough 5 character)

Wrong email format (not enough 3 number)

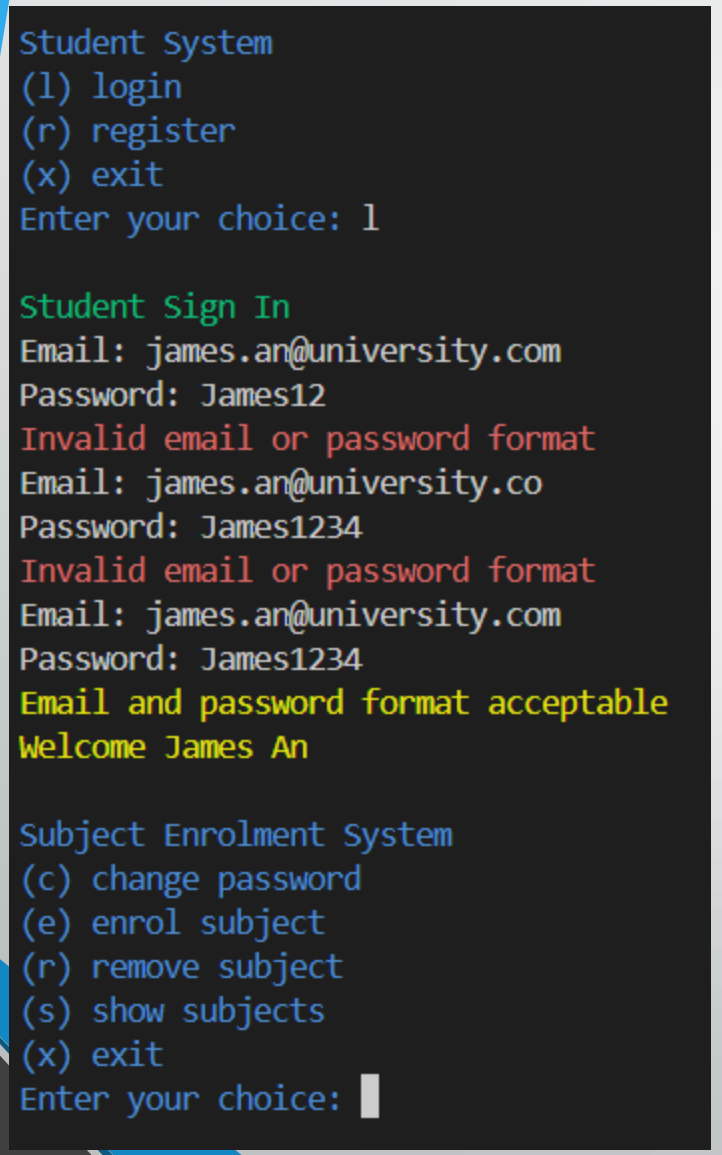
Successful registration

RESULT – Register Student

```
Enrolling Student James An

Student System
(l) login
(r) register
(x) exit
Enter your choice: r

Student Sign Up
Name: James An
Email: james.an@university.com
Password: James1234
Student James An already exists
```

Duplicated error

RESULT – Student - Login



Wrong password format

Wrong email format

Acceptable

Enrollment system menu

RESULT – Student - Enrollment

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: s
Showing 0 subjects.

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: e
Enrolling in Subject-009
You are now enrolled in 1 out of 4 subjects

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: s

Showing 1 subjects.
Subject: 009 - Mark: 72 - Grade: C
```

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: e
Enrolling in Subject-693
You are now enrolled in 4 out of 4 subjects

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: s

Showing 4 subjects.
Subject: 009 - Mark: 72 - Grade: C
Subject: 474 - Mark: 25 - Grade: Z
Subject: 735 - Mark: 78 - Grade: D
Subject: 693 - Mark: 66 - Grade: C

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: e
Students are allowed to enrol in 4 subjects only
```

Max 4

RESULT – Student – Remove a subject

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: s

Showing 4 subjects.
Subject: 009 - Mark: 72 - Grade: C
Subject: 474 - Mark: 25 - Grade: Z
Subject: 735 - Mark: 78 - Grade: D
Subject: 693 - Mark: 66 - Grade: C

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: r
Remove Subject by ID: 008
You did not enroll in Subject-008
```

Wrong ID

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: r
Remove Subject by ID: 009
Dropping Subject-009
You are now enrolled in 3 out of 4 subjects

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: s

Showing 3 subjects.
Subject: 474 - Mark: 25 - Grade: Z
Subject: 735 - Mark: 78 - Grade: D
Subject: 693 - Mark: 66 - Grade: C
```

Correct ID

RESULT – Student – Change Password

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: c
Enter new password: James12
Confirm password: James12
Invalid password format
```

Wrong password format

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: c
Enter new password: james1234
Confirm password: james1234
Invalid password format
```

Wrong password format

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: c
Enter new password: James1234
Confirm password: James1234
Password changed successfully

Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: c
Enter new password: James12345
Confirm password: James12345
Password changed successfully
```

May use old password

RESULT – Student – Change Password

```
Subject Enrolment System
(c) change password
(e) enrol subject
(r) remove subject
(s) show subjects
(x) exit
Enter your choice: c
Enter new password: James0123
Confirm password: James012
Password does not match - Try again
Confirm password: James0124
Password does not match - Try again
Confirm password: James0123
Password changed successfully
```

Type confirm password until being matched with new password

RESULT – Admin – Show students

```
Enter your choice: S
Students List.
Manh Nguyen :: 100001 --> Email: manh@university.com
James An :: 100002 --> Email: james@university.com
John Smith :: 100003 --> Email: john@university.com
Yanfei Tao :: 100004 --> Email: yanfei@university.com
Santhosh Ramachandran :: 100005 --> Email: santhosh@university.com
Sophia Wilson :: 100006 --> Email: sophia@university.com
David Lee :: 100007 --> Email: david@university.com
Olivia Martin :: 100008 --> Email: olivia@university.com
James Anderson :: 100009 --> Email: james@university.com
Isabella Thomas :: 100010 --> Email: isabella@university.com
Norman Nguyen :: 199768 --> Email: NormanNguyen@university.com
James An :: 301152 --> Email: james.an@university.com
```

RESULT – Admin – Group students

```
Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: g

Grouped Students

HD:
No students found.

D:
Sophia Wilson :: 100006 --> Grade: D - Mark: 80.33

C:
Manh Nguyen :: 100001 --> Grade: C - Mark: 65.25
Isabella Thomas :: 100010 --> Grade: C - Mark: 65.50

P:
James An :: 100002 --> Grade: P - Mark: 53.50
Yanfei Tao :: 100004 --> Grade: P - Mark: 60.00
James An :: 301152 --> Grade: P - Mark: 56.33

Z:
John Smith :: 100003 --> Grade: Z - Mark: 0.00
Santhosh Ramachandran :: 100005 --> Grade: Z - Mark: 0.00
David Lee :: 100007 --> Grade: Z - Mark: 0.00
Olivia Martin :: 100008 --> Grade: Z - Mark: 43.50
```

RESULT – Admin – Partition students

```
Enter your choice: P

PASS/FAIL Partition

PASS:
Manh Nguyen :: 100001 --> Grade: C - Mark: 65.25
James An :: 100002 --> Grade: P - Mark: 53.50
Yanfei Tao :: 100004 --> Grade: P - Mark: 60.00
Sophia Wilson :: 100006 --> Grade: D - Mark: 80.33
Isabella Thomas :: 100010 --> Grade: C - Mark: 65.50
James An :: 301152 --> Grade: P - Mark: 56.33

FAIL:
John Smith :: 100003 --> Grade: Z - Mark: 0.00
Santhosh Ramachandran :: 100005 --> Grade: Z - Mark: 0.00
David Lee :: 100007 --> Grade: Z - Mark: 0.00
Olivia Martin :: 100008 --> Grade: Z - Mark: 43.50
James Anderson :: 100009 --> Grade: Z - Mark: 0.00
Norman Nguyen :: 199768 --> Grade: Z - Mark: 0.00
```

RESULT – Admin – Remove a student

```
James Anderson :: 100009 --> Email: james@university.com
Isabella Thomas :: 100010 --> Email: isabella@university.com
Norman Nguyen :: 199768 --> Email: NormanNguyen@university.com
James An :: 301152 --> Email: james.an@university.com
```

```
Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: r
Enter student ID: 301153
Student not found
```

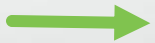
Not found ID

```
Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: r
Enter student ID: 301152
Removed student James An :: 301152

Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: s
Students List.
Manh Nguyen :: 100001 --> Email: manh@university.com
James An :: 100002 --> Email: james@university.com
John Smith :: 100003 --> Email: john@university.com
Yanfei Tao :: 100004 --> Email: yanfei@university.com
Santhosh Ramachandran :: 100005 --> Email: santhosh@university.com
Sophia Wilson :: 100006 --> Email: sophia@university.com
David Lee :: 100007 --> Email: david@university.com
Olivia Martin :: 100008 --> Email: olivia@university.com
James Anderson :: 100009 --> Email: james@university.com
Isabella Thomas :: 100010 --> Email: isabella@university.com
Norman Nguyen :: 199768 --> Email: NormanNguyen@university.com

Admin System
```

RESULT – Admin – Clear database

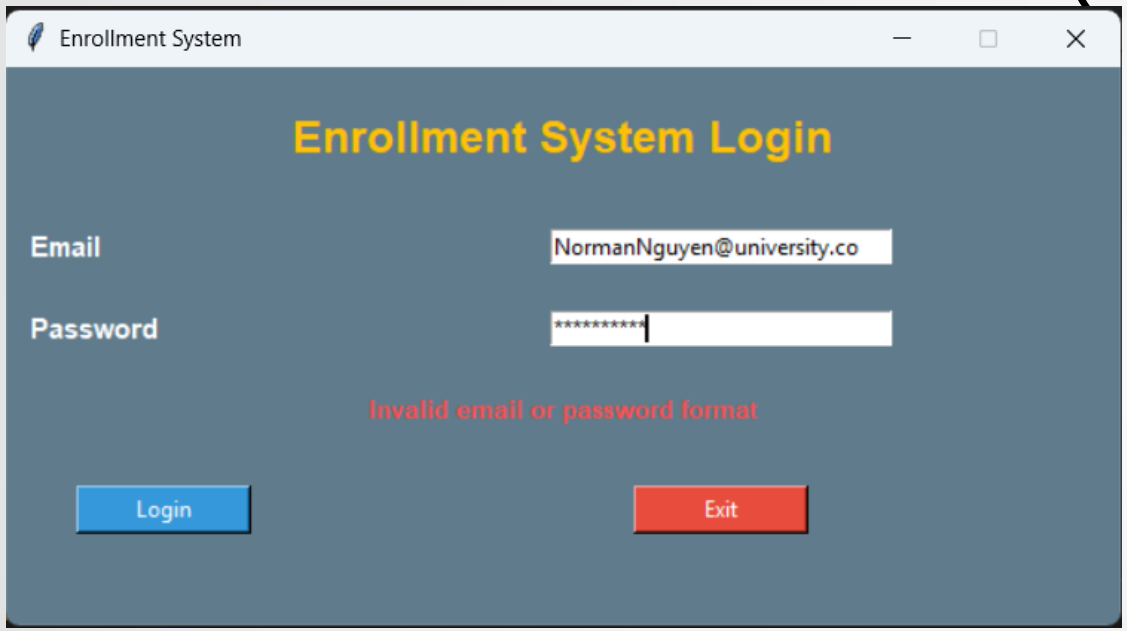
```
Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: c
Are you sure you want to clear all data? (y/n): n
Database clear cancelled

Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: c
Are you sure you want to clear all data? (y/n): y
Cleared database (11 students removed)

Admin System
(c) clear database
(g) group students
(p) partition students
(r) remove student
(s) show students
(x) exit
Enter your choice: s
No students found.
```

Cancled clearance

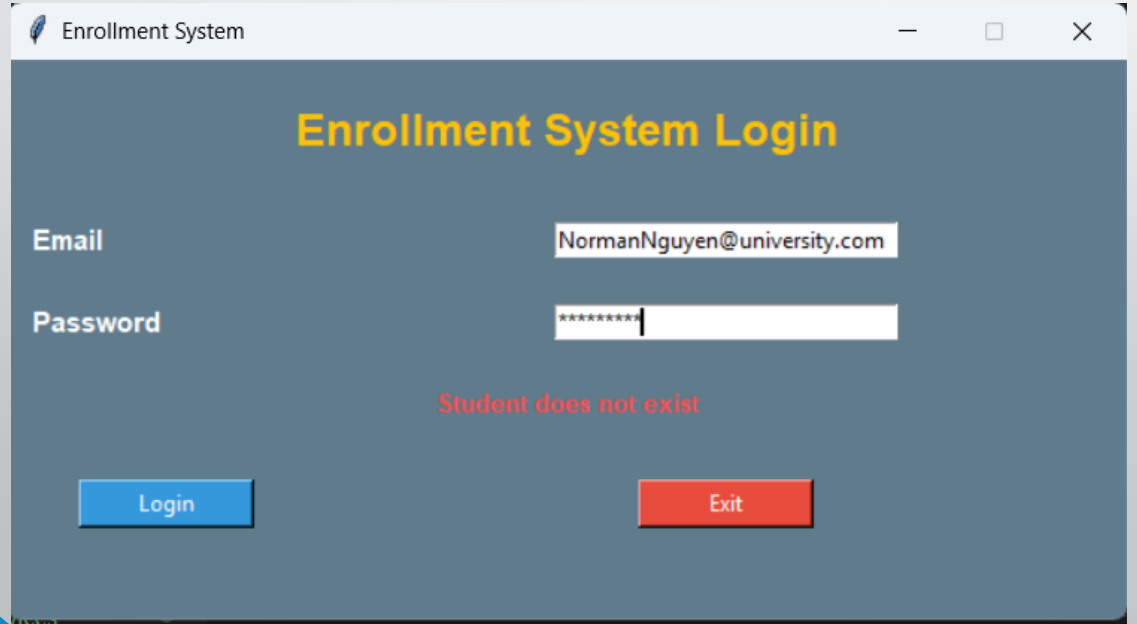


Successful clearance

RESULT – Student (GUI) - Login

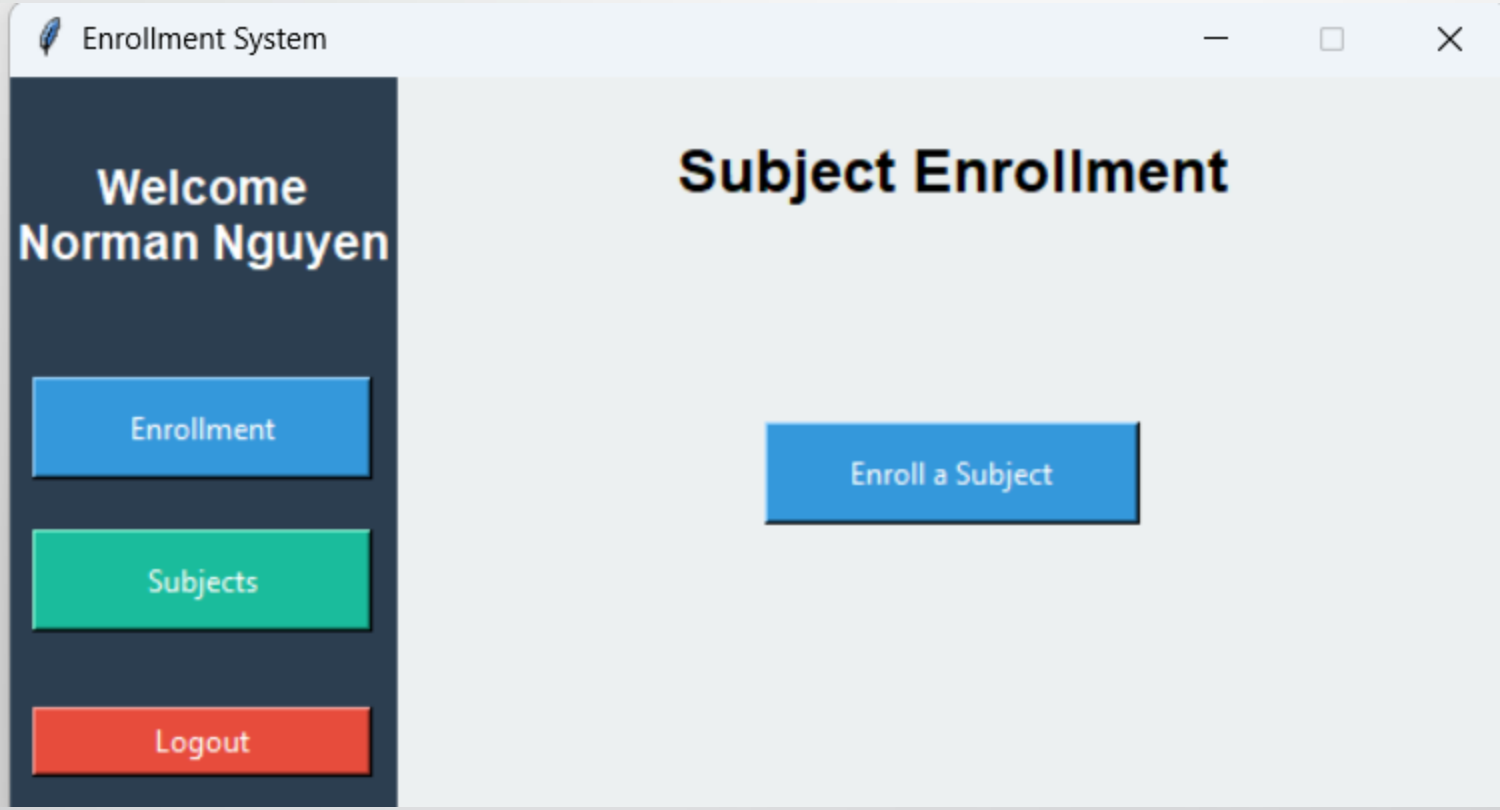


Wrong format



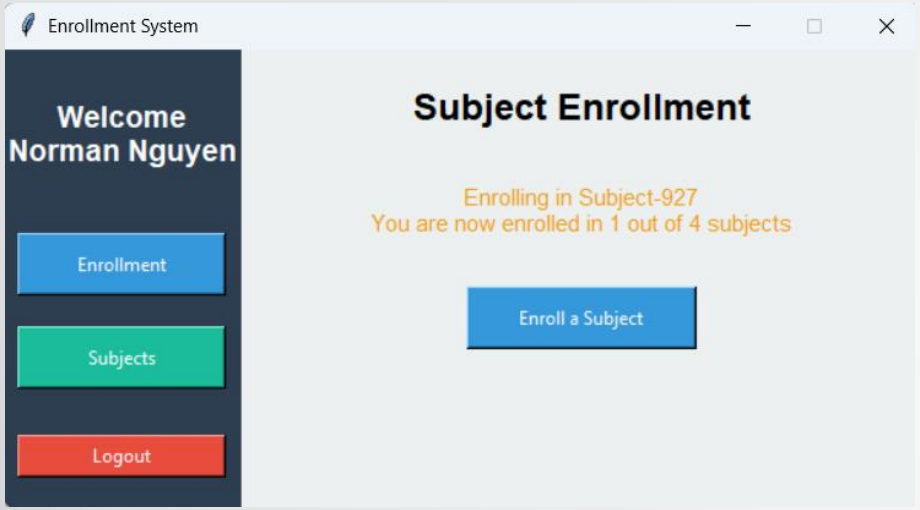
Wrong password or email

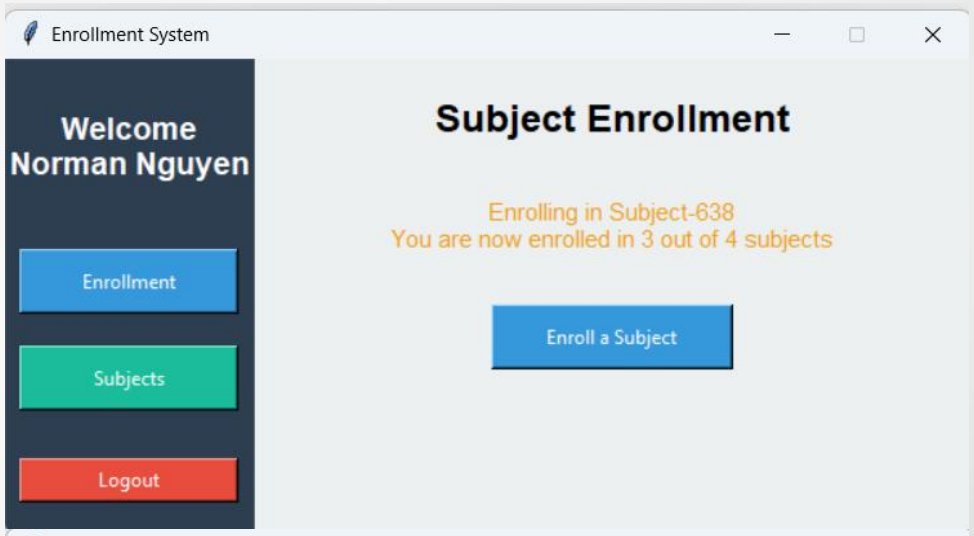
RESULT – Student (GUI) - Login

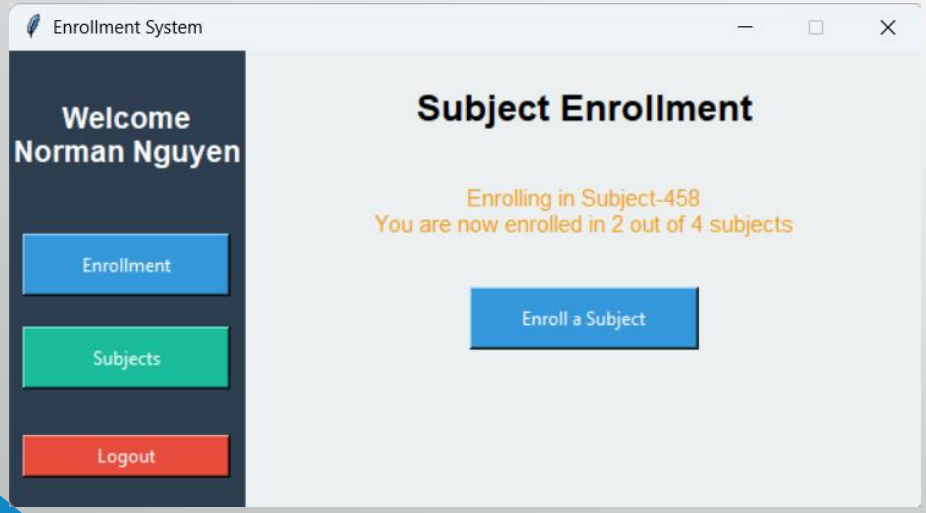


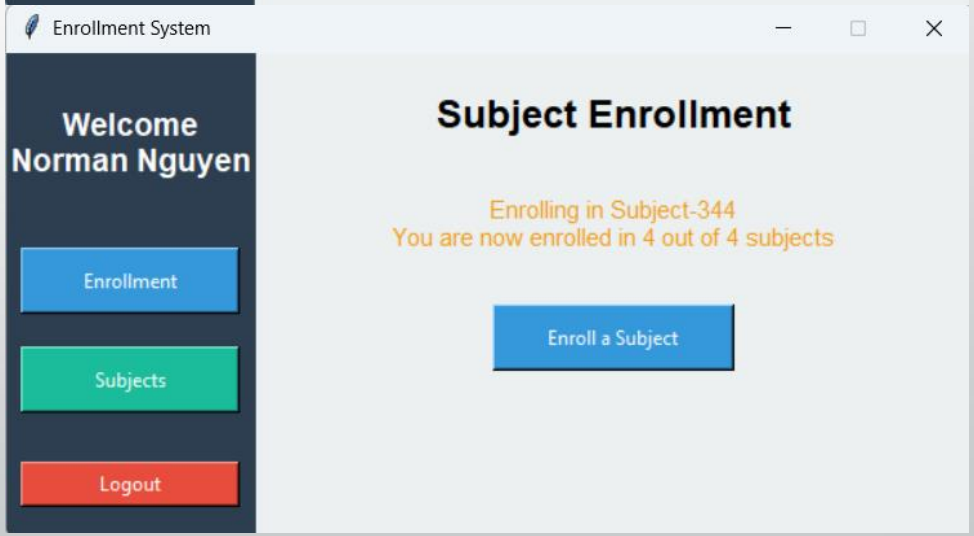
Login successfully

RESULT – Student (GUI) - Enrollment Window

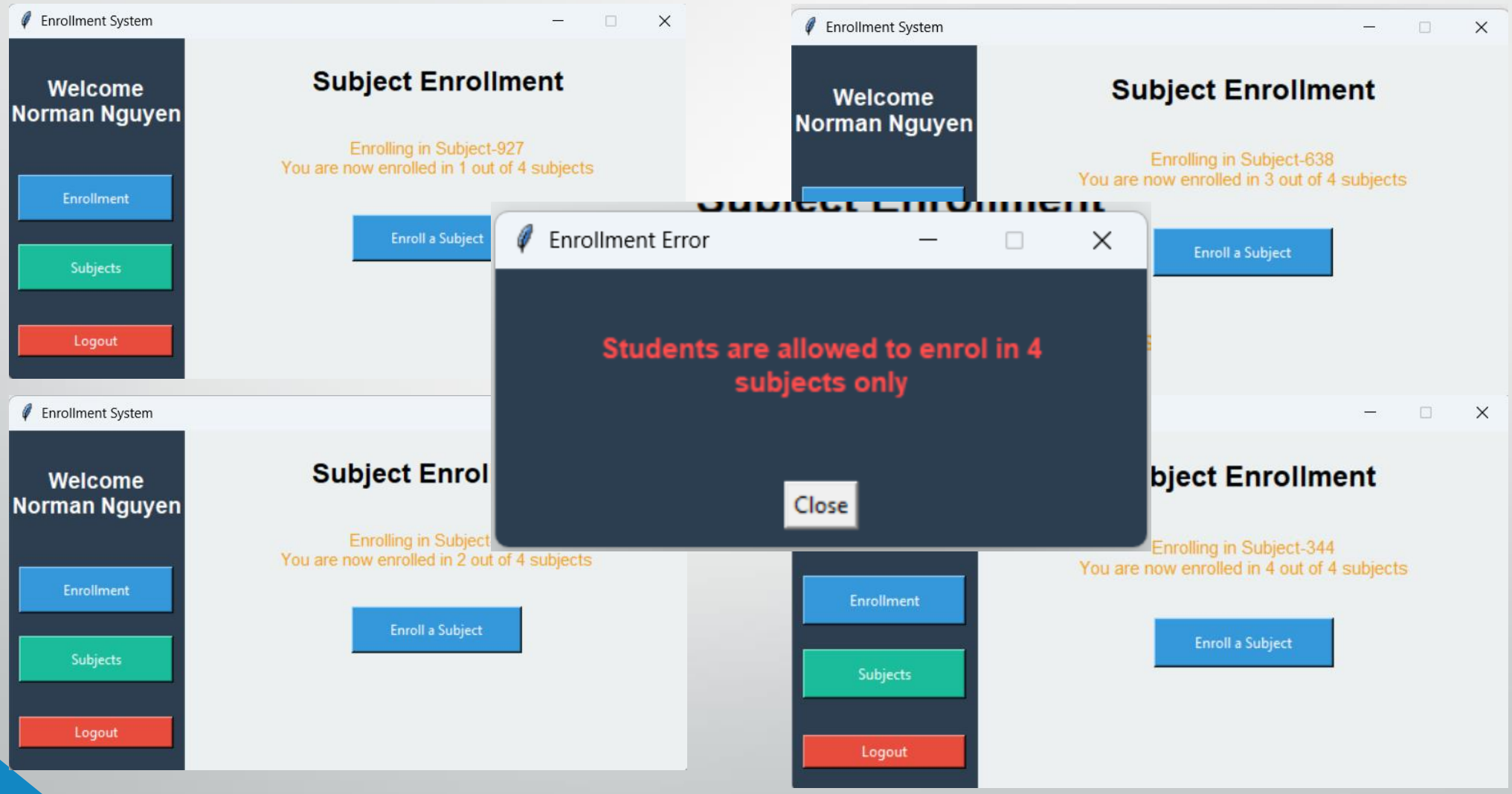




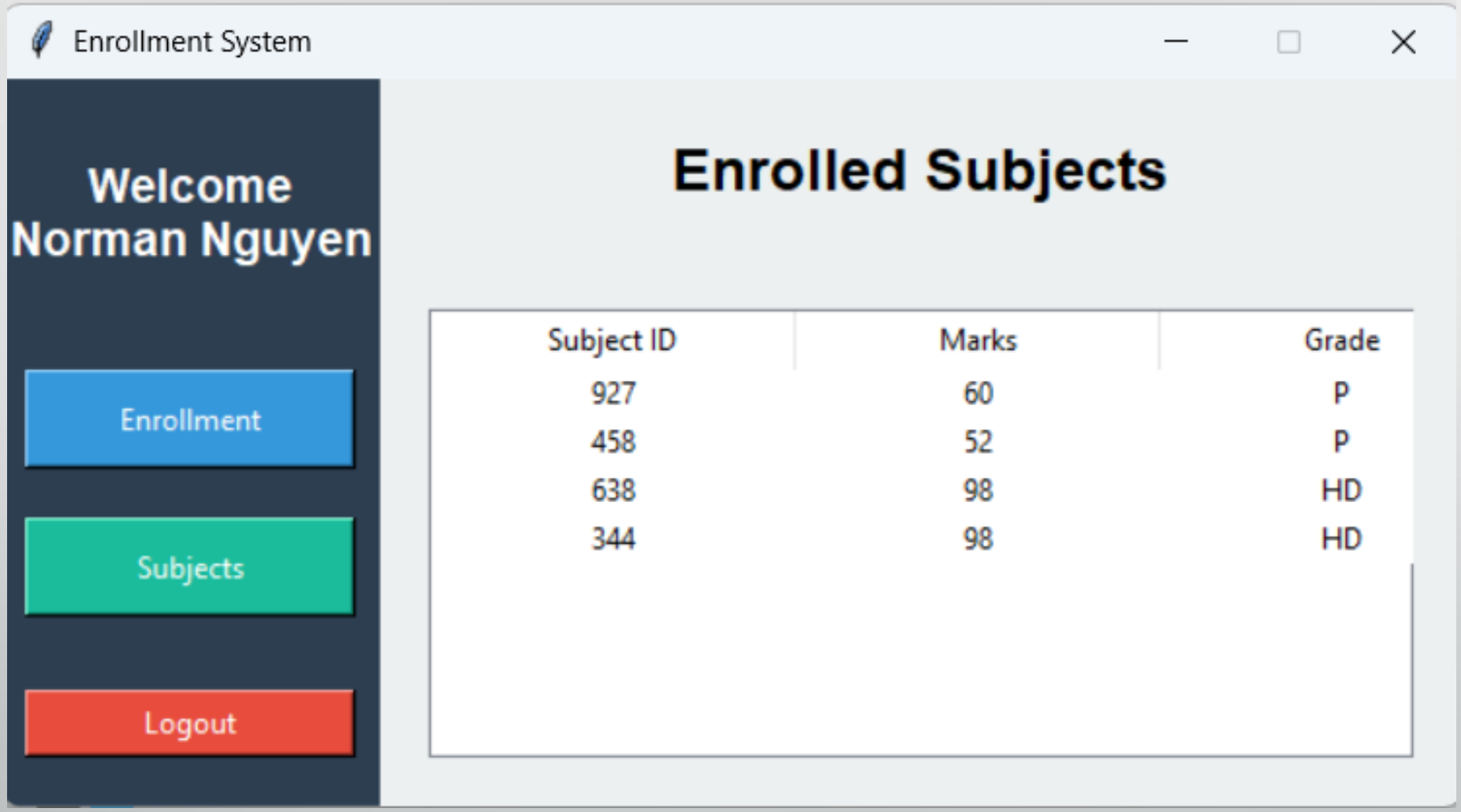




RESULT – Student (GUI) - Enrollment Window



RESULT – Student (GUI) - Subjects Window



Project Structure and Distribution

Layer	Class - Tasks	Member
View	CLI	James An
	GUI	Yanfei Tao
Controller	AuthController	James An
	AdminController	Santhosh Ramachandran
	StudentController	Ngoc Manh Nguyen
Models (Services)	Subjects	Yanfei Tao
	User	James An
	Student	Ngoc Manh Nguyen
	Admin	Santhosh Ramachandran
	DataManager	Ngoc Manh Nguyen Santhosh Ramachandran

Models - Subject

```python
import random

class Subject:
    def __init__(self, subject_id=None, mark=None):
        self.subject_ID = subject_id or self.generate_subject_id()
        self.mark = mark if mark is not None else self.generate_mark()
        self.grade = self.calculate_grade()

    def generate_subject_id(self):
        return str(random.randint(1, 999)).zfill(3)

    def generate_mark(self):
        # Generate random mark between 25 and 100
        return random.randint(25, 100)

    def calculate_grade(self):
        # Convert mark to grade (Z, P, C, D, HD)
        if self.mark < 50:
            return "Z"
        elif self.mark < 65:
            return "P"
        elif self.mark < 75:
            return "C"
        elif self.mark < 85:
            return "D"
        else:
            return "HD"
```

```python
    def get_subject_id(self):
        return self.subject_ID

    def get_marks(self):
        return self.mark

    def get_grade(self):
        return self.grade

    def __str__(self):
        return f"Subject: {self.subject_ID} - Mark: {self.mark} - Grade: {self.grade}"
```

Models - User

```python
import re

class User:
    EMAIL_PATTERN = (
        r"^[a-zA-Z0-9._%+-]+"
        r"@university\.com$"
    )

    PASSWORD_PATTERN = (
        r"^[A-Z][a-zA-Z]{5,}\d{3,}$"
    )

    def __init__(self, email, password):
        self._email = email
        self._password = password

    def validate_email(self):
        return re.fullmatch(
            self.EMAIL_PATTERN,
            self._email
        ) is not None

    def validate_password(self):
        return re.fullmatch(
            self.PASSWORD_PATTERN,
            self._password
        ) is not None
```

Models - Student

```python
from models.user import User
from models.subject import Subject

import random


class Student(User):
    def __init__(self, email, password, name, student_id=None, enrolled_subjects=None):
        super().__init__(email, password)
        self.name = name
        self.student_id = student_id if student_id is not None else self.autogenerate_student_id()
        if enrolled_subjects is None:
            self.enrolled_subjects = []
        else:
            self.enrolled_subjects = enrolled_subjects

    def autogenerate_student_id(self):
        return str(random.randint(1, 999999)).zfill(6)

    def enroll_subject(self, subject=None):
        if self.check_Subject_limit():
            return None
        if subject is None:
            subject = Subject()
        self.enrolled_subjects.append(subject)
        return subject.get_subject_id()
```

```python
def check_Subject_limit(self):
    if len(self.enrolled_subjects) >= 4:
        return True
    return False
```

Models - Student

```python
def remove_subject(self, subject_id):
    subject = self.lookup_subject(subject_id)
    if subject is None:
        return None
    self.enrolled_subjects.remove(subject)
    return subject

def change_password(self, new_password):
    old_password = self._password
    self._password = new_password
    if self.validate_password():
        return True
    self._password = old_password
    return False
```

```python
def calculate_average_mark(self):
    if not self.enrolled_subjects:
        return 0
    total = 0
    for subject in self.enrolled_subjects:
        total += subject.get_marks()
    return total / len(self.enrolled_subjects)

def calculate_average_grade(self):
    average = self.calculate_average_mark()
    if average < 50:
        return "Z"
    elif average < 65:
        return "P"
    elif average < 75:
        return "C"
    elif average < 85:
        return "D"
    return "HD"

def is_passed(self):
    if self.calculate_average_mark() < 50:
            return False
    return True
```

Models - Admin

```python
from models.user import User

class Admin(User):
    def __init__(self, email, password):
        super().__init__(email, password)
        self.name = "Admin"

    def view_students(self, data_manager):
        return data_manager.students

    def organize_by_grade(self, data_manager):
        grouped_students = {
            "HD": [],
            "D": [],
            "C": [],
            "P": [],
            "Z": []
        }
        students = data_manager.get_all_students()
        for student in students:
            grade = (student.calculate_average_grade())
            grouped_students[grade].append(student)
        return grouped_students
```

```python
    def categorize_students(self, data_manager):
        categorized_students = {
            "PASS": [],
            "FAIL": []
        }
        students = data_manager.students
        for student in students:
            if student.is_passed():
                categorized_students["PASS"].append(student)
            else:
                categorized_students["FAIL"].append(student)
        return categorized_students

    def remove_student(self, student_id, data_manager):
        data_manager.remove_student(student_id)
        return True

    def clear_all_data(self, data_manager):
        data_manager.clear_all_data()
        return True
```

Models - DataManager

```python
class DataManager:
    def __init__(self):
        self.file_path = (PROJECT_ROOT / "data" / "students.data")
        self.students = []
        self.load_data()

    def load_data(self):
        self.students = []
        try:
            with open(self.file_path, "r") as f:
                data = json.load(f)
            for student_dict in data.get("students",[]):
                student = self.load_student_object(student_dict)
                self.students.append(student)
        except FileNotFoundError:
            raise FileNotFoundError("students.data file not found.")
        except json.JSONDecodeError:
            raise ValueError("Invalid JSON format in students.data")
        except Exception as e:
            raise RuntimeError(f"Unexpected error: {e}")
        return self.students


    def load_student_object(self, student_dict):
        enrolled_subjects = []
        for subject_dict in student_dict.get("subjects", []):
            subject = Subject(
                subject_id=subject_dict.get("subject_id"),
                mark=subject_dict.get("marks")
            )
            enrolled_subjects.append(subject)
        student = Student(
            email=student_dict.get("email"),
            password=student_dict.get("password"),
            student_id=student_dict.get("student_id"),
            name=student_dict.get("name"),
            enrolled_subjects=enrolled_subjects
        )
        return student
```

```python
def load_student_object(self, student_dict):
    enrolled_subjects = []
    for subject_dict in student_dict.get("subjects", []):
        subject = Subject(
            subject_id=subject_dict.get("subject_id"),
            mark=subject_dict.get("marks")
        )
        enrolled_subjects.append(subject)
    student = Student(
        email=student_dict.get("email"),
        password=student_dict.get("password"),
        student_id=student_dict.get("student_id"),
        name=student_dict.get("name"),
        enrolled_subjects=enrolled_subjects
    )
    return student
```

Models - DataManager

```python
def get_all_students(self):
    return self.students

def lookup_student_by_id(self, student_id):
    student_id = str(student_id).strip()
    for student in self.students:
        if (student.get_student_id() == student_id):
            return student
    return None

def lookup_student_by_email(self, email):
    for student in self.students:
        if student.get_email() == email:
            return student
    return None

def verify_credentials(self, email, password):
    student = self.lookup_student_by_email(email)
    if student is None:
        return None
    if student.get_password() == password:
        return student
    return None

def add_student(self, student):
    self.students.append(student)

def remove_student(self, student_id):
    student = self.lookup_student_by_id(student_id)
    if student is None:
        raise ValueError("Student not found")
    self.students.remove(student)
```

```python
    def add_student(self, student):
        self.students.append(student)

    def remove_student(self, student_id):
        student = self.lookup_student_by_id(student_id)
        if student is None:
            raise ValueError("Student not found")
        self.students.remove(student)

    def update_student(self, updated_student):
        for index, student in enumerate(self.students):
            if (student.get_student_id()==updated_student.get_student_id()):
                self.students[index] = (updated_student)
                return
        raise ValueError("Student not found")

    def clear_all_data(self):
        self.students = []
```

```python
def save_data(self):
    data = {"students": []}
    for student in self.students:
        student_dict = self.student_to_dict(student)
        data["students"].append(student_dict)
    with open(self.file_path, "w") as f:
        json.dump(data, f, indent=4)
```

Models - DataManager

```
def student_to_dict(self, student):
    subjects = []
    for subject in student.get_enrolled_subjects():
        subjects.append(self.subject_to_dict(subject))
    return {
        "student_id": student.get_student_id(),
        "name": student.get_name(),
        "email": student.get_email(),
        "password": student.get_password(),
        "subjects": subjects
    }

def save_data(self):
    data = {"students": []}
    try:
        for student in self.students:
            student_dict = (self.student_to_dict(student))
            data["students"].append(student_dict)
        with open(self.file_path, "w") as f:
            json.dump(data,f,indent=4)
    except Exception as e:
        raise RuntimeError(f"Error saving data: {e}")
```

Controller – AuthController

```python
class AuthController:
    def __init__(self, data_manager):
        self.data_manager = data_manager

    def register_student(self, name, email, password):
        student = Student(
            email=email,
            password=password,
            name=name
        )
        if not student.validate_email() or not student.validate_password():
            return {
                    "success": False,
                    "message": f"Invalid email or password format" }
        existing_student = self.data_manager.lookup_student_by_email(email)
        if existing_student:
            return {
                    "success": False,
                    "message": f"Student {existing_student.get_name()} already exists" }
        self.data_manager.add_student(student)
        self.data_manager.save_data()
        return {
                "success": True,
                "message":
                    f"\nEnrolling Student {student.get_name()}"
            }
```

Controller – AuthController

```
def login(self, email, password):
    temp_student = Student(email=email, password=password, name="temp" )
    if not temp_student.validate_email() or not temp_student.validate_password():
        return {
                "success": False,
                "message": "Invalid email or password format" }
    student = self.data_manager.verify_credentials(email, password)
    if student is None:
        return {
                "success": False,
                "message": "Student does not exist" }
    return {
            "success": True,
            "message": f"Email and password format acceptable\nWelcome {student.get_name()}",
            "student": student
            }
```

Controller – StudentController

```python
class StudentController:
    def __init__(self, data_manager):
        self.data_manager = data_manager

    def enroll_subject(self, student):
        if student.check_Subject_limit():
            return {
                "success": False,
                "message":
                    "Students are allowed to enrol in 4 subjects only"
            }
        subject_id = student.enroll_subject()
        self.data_manager.update_student(student)
        self.data_manager.save_data()
        enrolled_subject = student.get_enrolled_subjects()

        return {
                "success": True,
                "message":
                    f"Enrolling in Subject-{subject_id}"
                    f"\nYou are now enrolled in {len(enrolled_subject)} out of 4 subjects"
            }
```

```python
    def remove_subject(self, student, subject_id):
        removed_subject = student.remove_subject(subject_id)
        if removed_subject is None:
            return {
                "success": False,
                "message":
                    f"You did not enroll in Subject-{subject_id}"
            }
        self.data_manager.update_student(student)
        self.data_manager.save_data()
        enrolled_subject = student.get_enrolled_subjects()
        return {
                "success": True,
                "message":
                    f"Dropping Subject-{subject_id}"
                    f"\nYou are now enrolled in {len(enrolled_subject)} out of 4 subjects"
            }
```

```python
    def change_password(self, student, new_password):
        success = student.change_password(new_password)
        if success:
            self.data_manager.update_student(student)
            self.data_manager.save_data()
            return {
                "success": success,
                "message": "Password changed successfully"
            }
        return {
                "success": success,
                "message": "Invalid password format"
            }

    def show_subjects(self, student):
        return student.get_enrolled_subjects()
```

Controller – AdminController

```python
class AdminController:
    def __init__(self, data_manager):
        self.data_manager = data_manager

    def view_students(self, admin):
        students = admin.view_students(self.data_manager )
        result = []
        for student in students:
            result.append({
                            "name": student.get_name(),
                            "id": student.get_student_id(),
                            "email": student.get_email() })
        return result

    def organize_by_grade(self, admin):
        grouped_students = admin.organize_by_grade(self.data_manager)
        result = {}
        for grade, students in grouped_students.items():
            result[grade] = []
            for student in students:
                result[grade].append({
                                        "name": student.get_name(),
                                        "id": student.get_student_id(),
                                        "grade": student.calculate_average_grade(),
                                        "mark": student.calculate_average_mark() })
        return result
```

Controller – AdminController

```
def categorize_students(self, admin):
    categorized_students = admin.categorize_students(self.data_manager)
    result = {}
    for grade, students in categorized_students.items():
        result[grade] = []
        for student in students:
            result[grade].append({
                                  "name": student.get_name(),
                                  "id": student.get_student_id(),
                                  "grade": student.calculate_average_grade(),
                                  "mark": student.calculate_average_mark() })
    return result

def remove_student(self, admin, student_id):
    student = self.data_manager.lookup_student_by_id(student_id)
    if student is None:
        return {
                "success": False,
                "message": "Student not found"
                }
    admin.remove_student(student_id, self.data_manager)
    self.data_manager.save_data()
    return {
            "success": True,
            "message": f"Removed student {student.get_name()} :: {student.get_student_id()}" }
```

Controller – AdminController

```
def clear_all_data(self, admin):
    students = self.data_manager.get_all_students()
    total_students = len(students)
    admin.clear_all_data(self.data_manager)
    self.data_manager.save_data()
    return {
            "success": True,
            "message": f"Cleared database ({total_students} students removed)" }
```

View – CLI

```
class CLIUniApp:
    def __init__(self):
        self.data_manager = DataManager()
        self.auth_controller = AuthController(self.data_manager)
        self.student_controller = StudentController(self.data_manager)
        self.admin_controller = AdminController(self.data_manager)
        self.current_user = None

    def start(self):
        while True:
            print("\n\033[34mUniversity System")
            print("(A) Admin")
            print("(S) Student")
            print("(X) Exit")
            choice = input("Enter your choice: \033[0m").lower()
            match choice:
                case "a":
                    self.admin_menu()
                case "s":
                    self.student_menu()
                case "x":
                    print("\033[33mThank you!\033[0m")
                    break
                case _:
                    print("Invalid option.")
```

View – CLI

```python
def student_menu(self):
    while True:
        print("\n\033[34mStudent System")
        print("(l) login")
        print("(r) register")
        print("(x) exit")
        choice = input("Enter your choice: \033[0m").lower()
        match choice:
            case "l": self.handle_login()
            case "r": self.handle_register()
            case "x": break
            case _: print("Invalid option.")
```

```python
def handle_register(self):
    print("\nStudent Sign Up")
    while True:
        name = input("Name: ")
        email = input("Email: ")
        password = input("Password: ")
        result = self.auth_controller.register_student(name,email,password)
        if result["success"]:
            print(f"\033[33m{result['message']}\033[0m")
            break
        else:
            print(f"\033[31m{result['message']}\033[0m")


def handle_login(self):
    print("\n\033[32mStudent Sign In\033[0m")
    while True:
        email = input("Email: ")
        password = input("Password: ")
        result = self.auth_controller.login(email,password)
        if not result["success"]:
            print(f"\033[31m{result['message']}\033[0m")
            continue
        self.current_user  = result["student"]
        print(f"\033[33m{result['message']}\033[0m")
        self.subject_enrolment_menu()
        break
```

View – CLI

```python
def subject_enrolment_menu(self):
    while True:
        print("\n\033[34mSubject Enrolment System")
        print("(c) change password")
        print("(e) enrol subject")
        print("(r) remove subject")
        print("(s) show subjects")
        print("(x) exit")
        choice = input("Enter your choice: \033[0m").lower()
        match choice:
            case "c": self.change_password()
            case "e": self.enrol_subject()
            case "r": self.remove_subject()
            case "s": self.show_subjects()
            case "x":
                self.current_user = None
                break
            case _: print("Invalid option.")
```

```python
def change_password(self):
    new_password = input("Enter new password: ")
    confirm_password = input("Confirm password: ")
    while new_password != confirm_password:
        print(f"\033[31mPassword does not match - Try again\033[0m")
        confirm_password = input("Confirm password: ")
    result = self.student_controller.change_password(self.current_user,new_password)
    if result["success"]:
        print(f"\033[33m{result['message']} \033[0m")
    else:
        print(f"\033[31m{result['message']} \033[0m")
```

```python
def enrol_subject(self):
    result = self.student_controller.enroll_subject(self.current_user)
    if result["success"]:
        print(f"\033[33m{result['message']}\033[0m")
    else:
        print(f"\033[31m{result['message']}\033[0m")

def remove_subject(self):
    input_subject_id = input("Remove Subject by ID: ")
    result = self.student_controller.remove_subject(self.current_user, input_subject_id)
    if result["success"]:
        print(f"\033[33m{result['message']}\033[0m")
    else:
        print(f"\033[31m{result['message']}\033[0m")

def show_subjects(self):
    subjects = self.student_controller.show_subjects(self.current_user)
    if not subjects:
        print("\033[33mShowing 0 subjects.\033[0m")
        return
    print(f"\n\033[33mShowing {len(subjects)} subjects.\033[0m")
    for subject in subjects:
        print(subject)
```

View – CLI

```
def admin_menu(self):
    admin = Admin(
        email="admin@university.com",
        password="Admin123"
    )
    while True:
        print("\n\033[34mAdmin System")
        print("(c) clear database")
        print("(g) group students")
        print("(p) partition students")
        print("(r) remove student")
        print("(s) show students")
        print("(x) exit")
        choice = input("Enter your choice: \033[0m").lower()
        match choice:
            case "c": self.clear_database(admin)
            case "g": self.group_students(admin)
            case "p": self.partition_students(admin)
            case "r": self.remove_student(admin)
            case "s": self.show_students(admin)
            case "x": break
            case _: print("Invalid option.")
```

View – CLI

```python
def show_students(self, admin):
    students = self.admin_controller.view_students(admin)
    if not students:
        print("\033[31mNo students found.\033[0m")
        return
    print("\033[33mStudents List.\033[0m")
    for student in students:
        print(f"{student['name']} :: {student['id']} --> Email: {student['email']}")


def group_students(self, admin):
    grouped_students = self.admin_controller.organize_by_grade(admin)
    print("\n\033[33mGrouped Students\033[0m")
    for grade, students in grouped_students.items():
        print(f"\n{grade}:")
        if not students:
            print("No students found.")
            continue
        for student in students:
            print(f"{student['name']} :: {student['id']} --> Grade: {student['grade']} - Mark: {student['mark']:.2f}")
```

View – CLI

```
def partition_students(self, admin):
    partitioned_students = self.admin_controller.categorize_students(admin)
    print("\n\033[33mPASS/FAIL Partition\033[0m")
    for group, students in partitioned_students.items():
        print(f"\n{group}:")
        if not students:
            print("No students found.")
            continue
        for student in students:
            print(f"{student['name']} :: {student['id']} --> Grade: {student['grade']} - Mark: {student['mark']:.2f}")


def remove_student(self, admin):
    student_id = input("Enter student ID: ")
    result = self.admin_controller.remove_student(admin,student_id)
    if result["success"]:
        print(f"\033[33m{result['message']} \033[0m")
    else:
        print(f"\033[31m{result['message']} \033[0m")


def clear_database(self, admin):
    confirm = input("\033[31mAre you sure you want to clear all data? (y/n): \033[0m").lower()
    if confirm != "y":
        print(f"\033[33mDatabase clear cancelled\033[0m")
        return
    result = self.admin_controller.clear_all_data(admin)
    print(f"\033[31m{result['message']} \033[0m")
```

View – GUI – Login window

```python
import tkinter as tk
from gui_app.home_window import HomeWindow


class LoginWindow(tk.Frame):
    def __init__(self,master,auth_controller,student_controller):
        super().__init__(master)
        self.master = master
        self.auth_controller = auth_controller
        self.student_controller = student_controller
        self.configure(bg="#607b8b")
        self.columnconfigure(0, weight=1)
        self.columnconfigure(1, weight=3)
        self.place(relx=0.5,rely=0.5,anchor="center")
        title = tk.Label(
            self,
            text="Enrollment System Login",
            font=("Helvetica", 18, "bold"),
            fg="#ffc107",
            bg="#607b8b"
        )
        title.grid(row=0,column=0,columnspan=2,pady=20)
        email_label = tk.Label(
            self,
            text="Email",
            fg="white",
            bg="#607b8b",
            font=("Helvetica", 11, "bold")
```

```python
def login_action(self):
    email = self.email_entry.get()
    password = self.password_entry.get()
    result = self.auth_controller.login(email,password)
    if not result["success"]:
        self.message_label.config(text=result["message"])
        return
    # Clear current frame
    self.destroy()

    # Open Home Window
    home_window = HomeWindow(self.master,result["student"],self.student_controller)
    home_window.pack(fill="both",expand=True)
```

View – GUI– Home window

```python
import tkinter as tk

from gui_app.enrollment_frame import EnrollmentFrame
from gui_app.subjects_frame import SubjectsFrame

class HomeWindow(tk.Frame):
    def __init__(self,master,student,student_controller):
        super().__init__(master)
        self.student = student
        self.student_controller = student_controller

        self.configure(bg="#ecf0f1")

        # Sidebar
        self.sidebar = tk.Frame(self,bg="#2c3e50",width=200)
        self.sidebar.pack(side="left",fill="y")

        # Welcome Label

        welcome_label = tk.Label(
            self.sidebar,
            text=(
                f"Welcome\n"
                f"{student.get_name()}"
            ),
            fg="white",
            bg="#2c3e50",
            font=("Helvetica", 14, "bold")
```

```python
class HomeWindow(tk.Frame):
    def __init__(self,master,student,student_controller):
        )
        logout_button.pack(side="bottom",pady=20)

        # Content Area
        self.content_frame = tk.Frame(self,bg="#ecf0f1")
        self.content_frame.pack(side="right",fill="both",expand=True)

        # Default View
        self.show_enrollment()

    def clear_content(self):
        for widget in (self.content_frame.winfo_children()):
            widget.destroy()

    # Enrollment View
    def show_enrollment(self):
        self.clear_content()
        enrollment_frame =EnrollmentFrame(self.content_frame,self.student,self.student_controller)
        enrollment_frame.pack(fill="both",expand=True)
    # Subjects View
    def show_subjects(self):
        self.clear_content()
        subjects_frame =SubjectsFrame(self.content_frame,self.student,self.student_controller)
        subjects_frame.pack(fill="both",expand=True)
    def logout(self):
        self.master.destroy()
```

View – GUI– Enrollment window

```python
from gui_app.exception_window import ExceptionWindow


class EnrollmentFrame(tk.Frame):

    def __init__(self,master,student,student_controller):
        super().__init__(master)
        self.student = student
        self.student_controller = student_controller
        self.configure(bg="#ecf0f1")
        title = tk.Label(
            self,
            text="Subject Enrollment",
            font=("Helvetica", 18, "bold"),
            bg="#ecf0f1"
        )
        title.pack(pady=20)
        self.status_label = tk.Label(
            self,
            text="",
            fg="#f39c12",
            bg="#ecf0f1",
            font=("Helvetica", 11)
        )
        self.status_label.pack(pady=10)
```

```python
def enroll_subject(self):
    result = (self.student_controller.enroll_subject(self.student))
    if not result["success"]:
        ExceptionWindow(self,"Enrollment Error",result["message"])
        return
    self.status_label.config(text=result["message"])
```

View – GUI– Exception window

```python
class StudentController:
    def __init__(self, data_manager):
        self.data_manager = data_manager

    def enroll_subject(self, student):
        if student.check_Subject_limit():
            return {
                "success": False,
                "message":
                    "Students are allowed to enrol in 4 subjects only"
            }
        subject_id = student.enroll_subject()
        self.data_manager.update_student(student)
        self.data_manager.save_data()
        enrolled_subject = student.get_enrolled_subjects()

        return {
                "success": True,
                "message":
                    f"Enrolling in Subject-{subject_id}"
                    f"\nYou are now enrolled in {len(enrolled_subject)} out of 4 subjects"
            }
```

View – GUI– Subject window

```python
import tkinter as tk
from tkinter import ttk

class SubjectsFrame(tk.Frame):
    def __init__(self,master,student,student_controller):
        super().__init__(master)
        self.student = student
        self.student_controller = student_controller
        self.configure(bg="#ecf0f1")

        # Title
        title = tk.Label(
            self,
            text="Enrolled Subjects",
            font=("Helvetica", 18, "bold"),
            bg="#ecf0f1"
        )
        title.pack(pady=20)


        # Table
        columns = ("Subject ID","Marks","Grade")
        self.tree = ttk.Treeview(self,columns=columns,show="headings",height=10)
        for column in columns:
            self.tree.heading(column,text=column)
            self.tree.column(column,width=150,anchor="center")
        self.tree.pack(fill="both",expand=True,padx=20,pady=20)
        self.load_subjects()
```

```python
# Load Subjects
def load_subjects(self):
    subjects = self.student_controller.show_subjects(self.student)
    for subject in subjects:
        self.tree.insert(
            "",
            "end",
            values=(
                subject.get_subject_id(),
                subject.get_marks(),
                subject.get_grade()
            )
        )
```